

Using Libraries in Java

(Example: File Operations and Understanding Different Operations)

Introduction

In Java, **libraries** are reusable collections of classes and methods. They let developers avoid “reinventing the wheel” by providing pre-built solutions for common tasks like **file handling, networking, collections, and concurrency**.

This note will cover:

- What libraries are and why they exist
 - File operations (create, read, write, append, delete)
 - Who implements library functions
 - How to find and debug into library source code
 - Shipping software: what to bundle and what not
 - Why libraries are called “**batteries included**”
 - Best practices in using and managing libraries
-

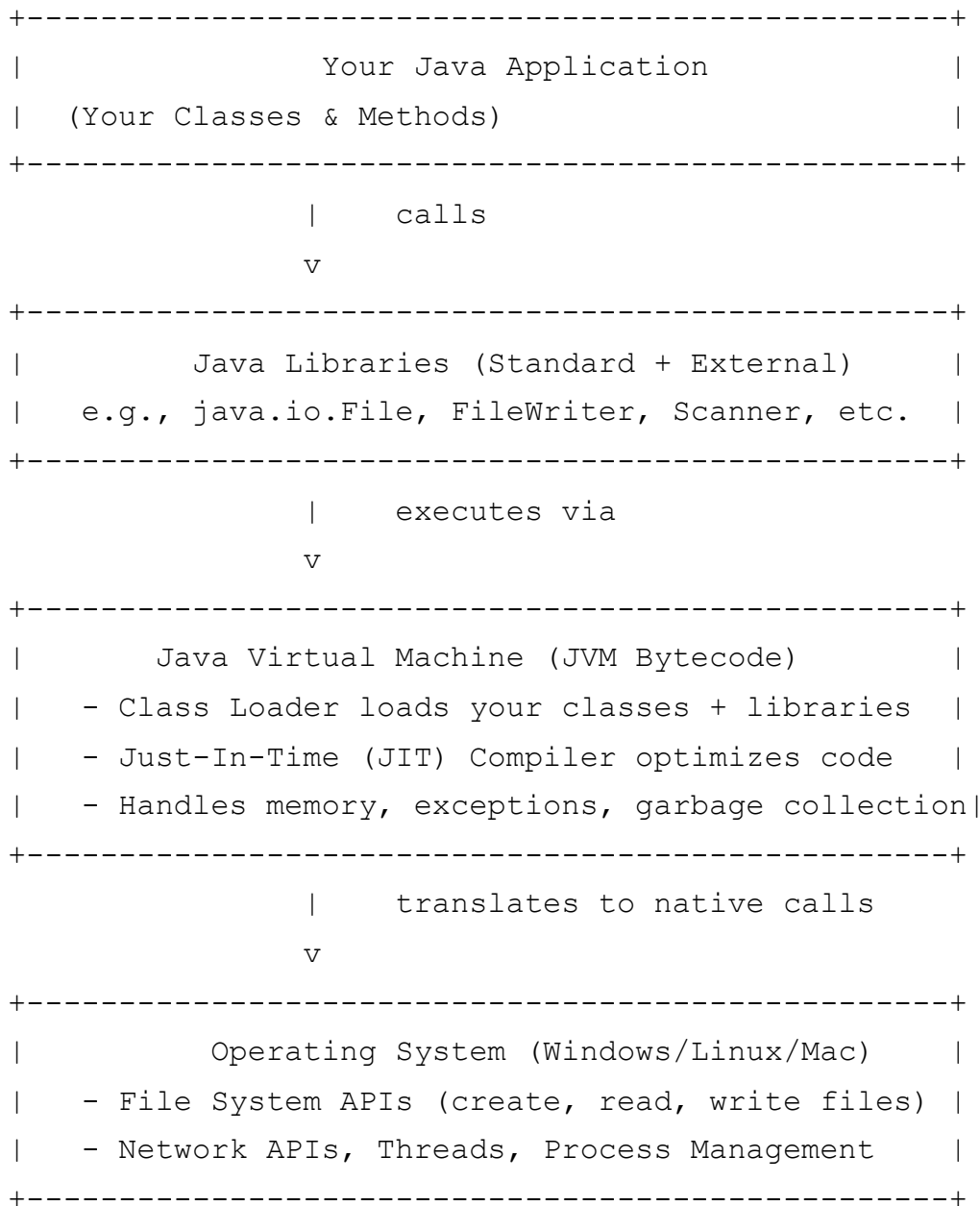
What is a Library in Java?

- A **library** is a collection of **packages** and **classes** with reusable functionality.
- In Java:
 - **Package** → Group of related classes (e.g., `java.io`, `java.util`).
 - **Class** → Defines attributes and methods (e.g., `File`, `Scanner`).
- Example Libraries:
 - **Standard Java Library**: `java.io`, `java.util`, `java.sql`, etc.
 - **Third-Party Libraries**: Apache Commons IO, Google Guava, Log4j.

Key Idea: Java ships with a **rich standard library**, but developers can also use external libraries.

Diagram – How Libraries Fit Into Java Execution

Here's a **textual diagram** to illustrate how your code interacts with libraries, JVM, and OS:



File Handling in Java (`java.io` package)

Common Classes

1. `File` → Represents a file or directory.
2. `FileReader` / `BufferedReader` → Reading text.
3. `FileWriter` / `BufferedWriter` → Writing text.
4. `PrintWriter` → Writing formatted text.
5. `Scanner` (from `java.util`) → Alternative for reading files.

File Operations Examples

1. Creating a File

```
import java.io.File;
import java.io.IOException;

public class CreateFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2. Writing to a File

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteFileExample {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("example.txt")) {
            writer.write("Hello, Java File Handling!\n");
            writer.write("This is a new line of text.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

3. Reading a File

```

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            Scanner reader = new Scanner(file);
            while (reader.hasNextLine()) {
                System.out.println(reader.nextLine());
            }
            reader.close();
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

4. Appending to a File

```

import java.io.FileWriter;
import java.io.IOException;

public class AppendFileExample {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("example.txt", true)) {
            writer.write("\nAppended line of text.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

5. Deleting a File

```

import java.io.File;

public class DeleteFileExample {
    public static void main(String[] args) {
        File file = new File("example.txt");
    }
}

```

```
        if (file.delete()) {
            System.out.println("Deleted: " + file.getName());
        } else {
            System.out.println("Failed to delete.");
        }
    }
}
```

Who Implements These Library Functions?

- **Standard Libraries:** Written and maintained by the **OpenJDK team** (Oracle + contributors).
- **Third-Party Libraries:** Written by open-source communities (e.g., Apache Foundation, Google) or companies.
- **Developers:** You can also create custom libraries (JARs) and share them.

These functions are not “magic”; they’re just Java classes written by other programmers, tested, and bundled for reuse.

Finding Library Source Code

1. JDK Standard Library

- Source code is open and part of **OpenJDK**.
- Local JDK installation usually has `src.zip` → contains `.java` files.
- Example path: `/usr/lib/jvm/java-17-openjdk/src.zip`

2. Third-Party Libraries

- If open source → available on GitHub/GitLab.
 - If using **Maven/Gradle** → download `*-sources.jar`.
-

Can We Debug into Library Code?

Yes

Steps in IntelliJ IDEA / Eclipse:

1. Attach `src.zip` or `*-sources.jar`.
2. Place a breakpoint in your code.
3. Step into a library method (e.g., `FileReader.read()`).
4. IDE will show the library's source.

Limitation: If source not available, you can still debug bytecode, but variable names may be missing.

Shipping Software – Do We Ship Libraries Too?

Library Type	Ship It?
Java Standard Library (<code>java.io</code> , <code>java.util</code> , etc.)	No – it comes with the JDK/JRE.
Third-Party Libraries	Yes – must bundle them (Uber/Fat JAR, libs folder).
Custom Libraries	Yes – include your own <code>.jar</code> .

Ways to Bundle

- **Fat/Uber JAR** → all code + dependencies in one `.jar`.
- **Dependencies folder** → keep separate `.jar` files.
- **Self-contained distribution** → ship with a private JRE.

Use Maven/Gradle to manage and package dependencies properly.

Purpose of Libraries – “Batteries Included”

- Libraries are meant to:
 - Save time (no need to re-implement).
 - Improve reliability (tested code).
 - Ensure consistency (same implementation everywhere).
- Java's standard library is often called “**batteries included**” → covers **collections, IO, networking, security, multithreading, math**, etc.

Who Writes Library Source Code?

- **JDK Source** → Maintained by OpenJDK developers (Oracle, Red Hat, community).
- **Apache, Google, etc.** → Large open-source organizations maintain reusable libraries.
- **You** → Anyone can write libraries. You package them as `.jar` files and publish to Maven Central or private repos.

To see source:

- Visit [OpenJDK GitHub Repo](#)
- Visit GitHub/GitLab projects for third-party libraries.

Comparison with Other Languages

Operation	Java (<code>java.io</code>)	Python (<code>os</code> , <code>open</code>)	C (<code>stdio.h</code>)
Create	<code>File.createNewFile()</code>	<code>open("file.txt", "w")</code>	<code>fopen("file.txt", "w")</code>
Read	<code>Scanner</code> / <code>FileReader</code>	<code>open("file.txt", "r")</code>	<code>fopen("file.txt", "r")</code>
Write	<code>FileWriter</code>	<code>open("file.txt", "w")</code>	<code>fprintf()</code>
Append	<code>FileWriter("file.txt", true)</code>	<code>open("file.txt", "a")</code>	<code>fopen("file.txt", "a")</code>
Delete	<code>file.delete()</code>	<code>os.remove("file.txt")</code>	<code>remove("file.txt")</code>

Summary (Skill-Oriented Takeaways)

- Libraries are **reusable building blocks** created by experienced developers.
- Source code for Java standard library is open (OpenJDK). Attach source in IDE to debug into it.
- When shipping:
 - Standard library → already included in JDK.
 - Third-party/custom libraries → must be bundled.
- Libraries exist to provide a **"batteries included"** experience.
- Always use dependency managers (Maven/Gradle) to handle which libraries to include.
- Knowing how to trace into library source improves debugging and learning.

